Unidad 1: Introducción

1. SSDD — Introducción Parte 1

Computación Centralizada y sus límites

La **computación centralizada** concentra todos los recursos (cómputo, memoria, almacenamiento) en un único procesador administrado por un solo SO. Con el tiempo esta arquitectura llegó a sus límites físicos:

- Estancamiento en el incremento de la frecuencia de reloj (mayor frecuencia → mayor consumo y temperatura).
- Límite de integración de transistores por chip.
- Restricciones de memoria y prestaciones generales.

Grand Challenge Problems y motivaciones

Los *Grand Challenge Problems* son problemas científicos o de ingeniería que requieren arquitecturas especiales para obtener tiempos de respuesta adecuados. Se clasifican en dos grandes tipos:

- **Problemas complejos**: simulaciones, predicciones, modelado, renderizaciones. Requieren gran poder de cómputo.
- **Big Data**: conjuntos de datos que el hardware/software tradicional no puede percibir, gestionar ni procesar en tiempo razonable. En 2001, Doug Laney (Gartner) propuso las *Tres V* (Volumen, Velocidad, Variedad) para caracterizarlo.

Las demandas actuales incluyen mayor poder de cómputo, flexibilidad, acceso remoto y aplicaciones más sofisticadas.

High-Performance Computing (HPC)

HPC es la aplicación de supercomputadoras a problemas computacionales demasiado grandes o lentos para computadoras estándar. Sus dos paradigmas principales son:

Paradigma	Objetivo principal	Acoplamiento
Sistemas Paralelos (SSPP)	Maximizar speedup de un único problema	Fuertemente acoplado
Sistemas Distribuidos (SSDD)	Permitir cooperación entre múltiples usuarios	Débilmente acoplado

Características de cada paradigma

Computación Distribuida enfatiza:

- Múltiples recursos físicamente dispersos.
- Múltiples aplicaciones de diferentes usuarios corriendo simultáneamente.
- Sin memoria compartida a nivel de hardware.

Computación Paralela enfatiza:

- Una única aplicación dividida en subtareas fuertemente acopladas.
- Objetivo: maximizar la velocidad de procesamiento de esa aplicación.

Concurrencia vs. Paralelismo vs. Distribución

- **Concurrencia**: múltiples tareas *en proceso* en un momento dado (hilos de control simultáneos).
- **Paralelismo real**: una tarea ejecutada en múltiples nodos al mismo tiempo (SSPP, fuertemente acoplados).
- **Sistema Distribuido**: múltiples tareas de múltiples usuarios ejecutándose simultáneamente (débilmente acoplados).

Los SSDD y SSPP *implican* concurrencia, pero la inversa no siempre se cumple.

Resumen de objetivos:

- SSDD → maximizar concurrencia y disponibilidad.
- SSPP → minimizar tiempos de respuesta y maximizar performance.

Arquitecturas: homogéneas, heterogéneas y memoria

- **Arquitectura Homogénea**: todos los nodos del mismo tipo.
- **Arquitectura Heterogénea**: nodos de diferente tipo (CPU, GPU, FPGA, etc.).
- **Computación Heterogénea**: usar distintos tipos de procesadores en un mismo sistema, cada uno haciendo lo que mejor sabe (CPU controla el programa, GPU procesa grandes volúmenes de datos, FPGA acelera algoritmos específicos).
- Paradigmas de memoria: **Memoria Compartida** vs. **Memoria Distribuida**.

---

2. SSDD — Introducción Parte 2

Distribuido vs. Descentralizado

	Centralizado	Descentralizado	Distribuido
Definición	Un único nodo controla todo	Procesos y recursos necesariamente dispersos (visión integradora)	Procesos y recursos *suficientemente* repartidos (visión expansiva)

Ejemplo	Mainframe	Blockchain, planta industrial	Gmail, CDN

El DNS es un caso ilustrativo: lógicamente centralizado, físicamente distribuido, y administrativamente descentralizado en varias organizaciones.

Definiciones formales

- **Liu (2004):** colección de computadoras independientes interconectadas en red, capaces de colaborar en una tarea sin memoria compartida.
- **Tanenbaum:** colección de elementos de cómputo autónomos que se presenta al usuario como un único sistema coherente.
- **Coulouris:** sistema donde los componentes se comunican y coordinan *solo* mediante paso de mensajes → esto genera tres características clave: **concurrencia**, **ausencia de reloj global** y **fallos independientes**.
- **Lamport (pesimista):** "un sistema distribuido es aquel en el que el fallo de una computadora que ni siquiera sabías que existía puede inutilizar tu propia computadora."

Ejemplos de sistemas distribuidos

- **Internet / Intranets**: la intranet es una porción de Internet administrada de forma separada, generalmente con firewall.
- **MMOGs** (Juegos masivos en línea): requieren respuesta en tiempo real y propagación de eventos a miles de jugadores simultáneos.
- **Computación móvil/ubicua**: dispositivos que cambian de ubicación continuamente (laptops, tablets, smartphones, wearables, embebidos).
- **Cloud Computing**: servicios como IaaS, PaaS, SaaS.
- **IoT**: integración de sensores y dispositivos cotidianos conectados a Internet.

Ventajas y desventajas

Ventajas: velocidad, confiabilidad, compartir datos y dispositivos, comunicación, flexibilidad, escalabilidad.

Desventajas: mayor superficie de ataque (seguridad), dependencia de la red, software complejo de interoperar, múltiples puntos de fallo.

Desafíos y complejidades

Los SSDD son complejos por:

- **Arquitectura**: múltiples organizaciones posibles.
- **Procesos**: clases y relaciones entre procesos.
- **Comunicación**: intercambio de datos confiable.
- **Coordinación**: algoritmos independientes de la aplicación.
- **Naming**: identificación de recursos.
- **Consistencia y replicación**: mantener réplicas coherentes.
- **Tolerancia a fallas**: seguir operando ante fallas parciales.
- **Seguridad**: acceso autorizado a recursos.

Transparencia

La transparencia busca ocultar que los procesos y recursos están físicamente distribuidos. Sus tipos:

Tipo	Qué oculta
Acceso	Diferencia entre acceso local y remoto
Ubicación	Dónde está el recurso
Concurrencia	Acceso concurrente a recursos compartidos
Replicación	Existencia de réplicas
Fallo	Ocurrencia y recuperación de fallos
Movilidad	Reubicación de recursos o clientes
Rendimiento	Reconfiguración según carga
Escalado	Expansión en tamaño

Heterogeneidad y Middleware

La heterogeneidad existe a nivel de redes, hardware, SO, lenguajes de programación e implementaciones. El **middleware** es la capa de software que provee abstracción y enmascara esa heterogeneidad, ofreciendo transparencia, portabilidad e interoperabilidad. Ejemplos: RPC, Java RMI, CORBA, ODBC, HTTP, AWS SNS, IBM MQ.

Extensibilidad y Escalabilidad

- **Extensibilidad**: capacidad de agregar/reemplazar componentes (hardware o software) sin rediseñar el sistema. Los SSDD abiertos permiten esto.
- **Escalabilidad**: efectividad conservada ante incremento significativo de recursos/usuarios.
    - *Escalabilidad de tamaño*: más usuarios o procesos.
    - *Escalabilidad geográfica*: mayor distancia entre nodos.

    - *Escalabilidad administrativa*: más dominios administrativos.
    - *Escalabilidad horizontal*: más servidores (clusters con balanceador de carga).
    - *Escalabilidad vertical*: más recursos en el servidor existente (CPU, RAM, almacenamiento).

Técnicas de escalado: ocultar latencias (comunicación asincrónica), particionamiento de datos y cómputo, replicación y caché.

Confianza (Dependability)

Grado en que se puede confiar que el sistema opera como se espera. Dimensiones:

- **Disponibilidad (Availability)**: listo para usar inmediatamente. Disponibilidad = Tiempo operativo / (Tiempo operativo + Tiempo de caída). Expresada en "nueves" (99%, 99.9%, 99.99%…).
- **Fiabilidad (Reliability)**: funciona continuamente sin fallos.
- **Seguridad (Safety)**: ante un fallo temporal, no ocurre ningún evento catastrófico.
- **Mantenibilidad (Maintainability)**: facilidad de reparación.

Métricas de tolerancia a fallas:

- MTTF: Mean Time To Failure.
- MTTR: Mean Time To Repair.
- MTBF = MTTF + MTTR.
- FIT: Failure In Time (fallos esperados por mil millones de horas).

Tipos de fallas:

- **Transitoria**: ocurre una vez y desaparece.
- **Intermitente**: aparece y desaparece repetidamente.
- **Permanente**: persiste hasta reemplazar el componente.

Tratamiento de fallos:

- Detección (checksums).
- Enmascaramiento (retransmisión, replicación).
- Tolerancia (diseño de programas resilientes).
- Recuperación (rollback a estado previo).
- Redundancia de componentes.

Seguridad

Tres componentes clave:

- **Confidencialidad**: protección ante acceso no autorizado.
- **Integridad**: protección ante alteración o corrupción.
- **Disponibilidad**: protección ante interferencia que impide el acceso.

Tipos de SSDD (Tanenbaum)

- **Sistemas de cómputo distribuido**: clusters, grids.
- **Sistemas distribuidos de información**: procesamiento de transacciones, integración empresarial (EAI).
- **Sistemas pervasivos**: ubicuos (IoT, sensores), computación móvil, redes de sensores.

Data Centers (clusters): instalaciones de computadoras en red para procesar y almacenar grandes volúmenes de datos. Ejemplos: Google, Facebook, Amazon (entre 50.000 y 80.000 servidores por centro).

Sistemas ubicuos se caracterizan por: distribución transparente, interacción alta, conocimiento del contexto, autonomía e inteligencia del sistema.

Redes de sensores (WSN): nodos simples, muchos (10s a 1000s), con limitaciones de memoria/cómputo/batería.

---

3. SSDD — Arquitecturas de Sistemas Distribuidos

Antecedentes: Ley de Moore y Ley de Metcalfe

- **Ley de Moore**: el rendimiento de los procesadores se duplica aproximadamente cada 18 meses. Desde 2005 esto no se mantiene en frecuencia; se compensó con múltiples núcleos y computación heterogénea (GPU, FPGA).
- **Ley de Metcalfe**: el valor de una red es proporcional al cuadrado del número de usuarios conectados. Explica el crecimiento exponencial del valor de plataformas como redes sociales.

Arquitecturas de Software

Las **arquitecturas de software** describen cómo se organizan e interactúan los componentes. La instanciación en máquinas reales da lugar a la **arquitectura de sistema**.

Capas de software en SSDD:

Aplicaciones / Servicios

Middleware ← Transparencia, portabilidad, interoperabilidad

Sistema Operativo

Plataforma (HW + SO básico)

- **Plataforma**: servicios de bajo nivel dependientes del HW y SO (ej. x86 Linux). Puede virtualizarse.
- **Middleware**: enmascara la heterogeneidad y provee abstracciones como RPC, RMI, comunicación en grupo, eventos, replicación.

Estilos Arquitectónicos (Tanenbaum)

Arquitectura en Capas

Cada capa se comunica solo con la capa adyacente mediante interfaces bien definidas. Variantes: modelo estricto (a), uso de capas no adyacentes (b), llamadas inversas —capa inferior llama a superior— (c).

Vista tradicional de 3 capas en aplicaciones distribuidas:

- Capa de interfaz de usuario.
- Capa de procesamiento (lógica de negocio).
- Capa de datos.

Arquitectura Orientada a Objetos / Servicios

Los objetos encapsulan datos y métodos. Los componentes (objetos) se conectan mediante llamadas a procedimientos que pueden ejecutarse a través de la red (RMI, CORBA).

Arquitectura Centrada en Recursos (REST)

- Los recursos se identifican con un esquema de nombre simple (URI).
- Interfaz uniforme: GET, PUT, DELETE, POST.
- Los servicios son *stateless* (sin estado).
- Respuestas en formato conocido (JSON, XML).

Arquitectura Basada en Eventos — Publicador/Suscriptor

- Comunicación asincrónica: desacopla remitentes de consumidores.
- El **publicador** envía mensajes a un canal de entrada; los **suscriptores** los reciben por canales de salida.
- Tecnologías: Apache Kafka, Amazon Kinesis, Google Pub/Sub, Microsoft Event Hubs.
- **Ventajas**: desacoplamiento, escalabilidad, confiabilidad, procesamiento diferido.
- **Limitaciones**: excesivo para sistemas simples, no apto para streaming A/V ni comunicación síncrona.

Colas de mensajes: mecanismo complementario al Pub/Sub en arquitecturas Cloud para desacoplar aplicaciones en bloques independientes.

Arquitecturas de Sistema (Coulouris)

Modelo Cliente-Servidor

El modelo más habitual (DNS, Web, FTP). Un servidor puede ser cliente de otro servicio.

Múltiples Servidores

Un servicio es provisto por varios servidores (replicación). Muy usado en DNS, Web y NIS.

Proxy y Caché

- **Caché**: almacena recursos frecuentes para respuesta más rápida.
- **Proxy web**: intermediario entre clientes y servidores web; aumenta disponibilidad.

Procesos de Igual a Igual (Peer-to-Peer)

Procesos con roles simétricos que realizan tareas semejantes. Útil para coordinación y algoritmos descentralizados.

Arquitecturas de Dos y Tres Niveles

- **Un nivel**: terminal tonta a mainframe.
- **Dos niveles**: cliente a un único servidor.
- **Tres niveles**: cada capa en una máquina separada.

Agentes Móviles

Programas en ejecución (código + datos) que se trasladan autónomamente entre computadoras en red para realizar tareas. Aplicaciones: monitoreo, descubrimiento, recuperación de información, gestión de redes.

Thin Clients

Clientes con procesamiento mínimo local; toda la lógica reside en el servidor. Hoy sustituidos en gran medida por smartphones.

Cloud Computing

Cuatro capas:

- **Hardware**: procesadores, routers, alimentación y refrigeración.
- **Infraestructura**: virtualización; gestión de almacenamiento y servidores virtuales (IaaS).
- **Plataforma**: abstracciones de almacenamiento y servicios (PaaS). Ej: Amazon S3.
- **Aplicación**: suites ofimáticas y aplicaciones reales (SaaS).

Arquitectura Edge-Server

Servidores situados en el borde de la red (límite empresa–Internet real). Razones para usar infraestructura perimetral:

- **Latencia y ancho de banda**: crítico para aplicaciones de tiempo real (AR/VR).
- **Fiabilidad**: garantías de conectividad extremas.
- **Seguridad y privacidad**: los datos sensibles no salen del perímetro local.

La *orquestación perimetral* implica decisiones de asignación de recursos, colocación de servicios y selección del nodo edge más adecuado (no siempre el más cercano).

Requisitos de Diseño de Arquitecturas Distribuidas

- **Rendimiento**: capacidad de respuesta (proxies, caché), productividad (jobs/tiempo), balanceo de carga.
- **Calidad de Servicio (QoS)**: reserva de recursos para flujos con requisitos temporales estrictos; rechaza solicitudes que no puedan cumplirse.
- **Fiabilidad**: que el sistema funcione correctamente.
- **Tolerancia a fallos**: redundancia de HW; retransmisión en protocolos.
- **Seguridad**: DMZ para acceso sin restricciones, confidencialidad, integridad, disponibilidad.

---

4. Modelos del Sistema (Coulouris, Cap. 2)

Los modelos del sistema proveen un marco formal para razonar sobre las propiedades de los sistemas distribuidos. Coulouris distingue tres dimensiones: **modelos físicos**, **modelos arquitectónicos** y **modelos fundamentales**.

4.1 Modelos Físicos

El modelo físico describe el hardware subyacente y la topología de red. Captura los tipos de computadoras y dispositivos que componen el sistema y su interconexión.

Generaciones de sistemas distribuidos

Primera generación (~1970s–1985): Internet todavía en etapa académica. Sistemas de entre 10 y 100 nodos conectados por LAN. Sin grandes preocupaciones de heterogeneidad ni movilidad.

Segunda generación (~1985–2000): Expansión de Internet. Aparición del modelo cliente-servidor. Miles a decenas de miles de nodos. Diversidad de plataformas (Windows, Unix, Mac).

Tercera generación (2000–presente): Cloud, mobile, IoT. Sistemas de millones a miles de millones de nodos. Diversidad extrema: desde sensores de 8-bit hasta supercomputadoras. Movilidad como caso central, no como caso borde.

Infraestructuras físicas típicas

Tipo	Características	Ejemplo
Cluster	Nodos homogéneos de alto rendimiento, red de baja latencia, ubicación física concentrada	Data center de Google
Grid	Nodos heterogéneos geográficamente distribuidos, administración federada	SETI@home, LHC GRID
Cloud	Nodos virtualizados, elásticos, administrados por el proveedor	AWS EC2, Azure
P2P	Nodos con roles simétricos, sin administración central	BitTorrent, blockchain
IoT/Embedded	Nodos con recursos mínimos, energía limitada, conexión inalámbrica	sensores industriales, wearables

Conectividad de red

- **LAN (Local Area Network)**: alta velocidad (1–100 Gbps), baja latencia (< 1 ms), administración única. Base de los clusters.
- **WAN (Wide Area Network)**: cobertura geográfica amplia, variable latencia (10 ms–200+ ms), menor ancho de banda efectivo que LAN. Conecta sites de una organización o el propio Internet.
- **Wireless / Mobile**: latencia variable, ancho de banda reducido respecto a cableada, problemas de cobertura y handoff. Crítica para computación ubicua.

- **Overlay networks**: redes lógicas construidas sobre la red física. Permiten enrutamiento alternativo, anonimato (Tor), distribución de contenido (CDN), etc.

4.2 Modelos Arquitectónicos

Un modelo arquitectónico define las entidades del sistema que se comunican entre sí, los roles que desempeñan y cómo están interconectadas. Las entidades son procesos o, en sistemas embebidos, threads dentro de un único proceso.

Comunicación entre entidades

Los patrones de comunicación que soporta la arquitectura incluyen:

- **IPC (Interprocess Communication)**: sockets, paso de mensajes. Nivel de abstracción más bajo.
- **Request-reply**: patrón cliente-servidor sobre IPC. La base de HTTP, DNS, etc.
- **RPC / RMI**: invocación remota que emula llamadas locales; oculta la red al programador.
- **Comunicación indirecta**: publicador/suscriptor, colas de mensajes, espacios de tuplas. El emisor no conoce al receptor.

Roles y responsabilidades

- **Cliente**: solicita servicios; generalmente inicia la comunicación.
- **Servidor**: provee servicios; generalmente espera pasivamente.
- **Peer**: rol simétrico; puede ser cliente y servidor simultáneamente.

La elección del patrón arquitectónico impacta directamente en la escalabilidad, tolerancia a fallos y complejidad del sistema.

Middleware

El middleware es la capa de software que se ubica entre el SO y las aplicaciones. Provee abstracciones de programación que enmascaran la heterogeneidad subyacente. Categorías:

- **Middleware de comunicación**: gestiona el transporte de mensajes (sockets, MQ).
- **Middleware transaccional**: asegura ACID sobre operaciones distribuidas (CORBA OTS, JTA).
- **Middleware de objetos**: expone objetos remotos (CORBA, Java RMI, COM).
- **Middleware de integración**: conecta aplicaciones heterogéneas (ESB, REST gateways).

4.3 Modelos Fundamentales

Los modelos fundamentales capturan las propiedades más abstractas del sistema, independientemente de su arquitectura concreta. Coulouris define tres:

Modelo de Interacción

Problemas del canal de comunicación:

- **Latencia**: retardo entre emisión y recepción.
- **Velocidad limitada** del canal.
- **Jitter**: fluctuación o cortes.
- **Deriva de relojes**: diferencias horarias entre nodos.

Dos familias de sistemas:

- **Síncronos**: tiempos de ejecución y transmisión acotados; relojes sincronizables. Permiten construir detectores de fallos confiables (si no responde dentro de T, falló).
- **Asíncronos**: sin cotas garantizadas; los eventos pueden observarse desordenados respecto a su generación (problema de causalidad). Son el caso general de Internet.

La mayoría de los algoritmos distribuidos deben funcionar en sistemas **asíncronos**, lo que los hace considerablemente más difíciles de diseñar. El resultado FLP (Fischer, Lynch, Paterson, 1985) prueba que en un sistema asíncrono con al menos un proceso que puede fallar, es *imposible* garantizar el consenso de manera determinista.

Modelo de Fallo

Clase de fallo	Afecta	Descripción
Fail-stop	Proceso	Se detiene; otros procesos lo detectan.
Crash	Proceso	Se detiene; otros pueden no detectarlo.
Omisión de canal	Canal	Mensaje en buffer de salida nunca llega al destino.
Omisión de envío	Proceso	Completa el send pero el mensaje no entra al buffer.
Omisión de recepción	Proceso	Mensaje en buffer de entrada nunca es recibido por el proceso.
Arbitrario / Bizantino	Proceso o canal	Comportamiento arbitrario: mensajes incorrectos, omisiones, pasos erróneos.

Fallas Bizantinas: nodos que actúan de forma irracional o maliciosa, transmitiendo información contradictoria. Causas: defectos de HW/SW, errores de red, configuraciones incorrectas, actores maliciosos. La **Tolerancia a Fallos Bizantinos (BFT)** permite que el sistema siga funcionando; el algoritmo pBFT (Liskov & Castro, ~1999) sentó las bases para tecnologías blockchain.

Construcción de servicios fiables a pesar de fallos:

- Enmascarar fallos con capas de software (TCP/IP sobre IP no fiable).
- Convertir fallos arbitrarios en fallos por omisión (checksums).
- Redundancia y replicación (RAID, múltiples servidores).

Modelo de Seguridad

El modelo de seguridad identifica las amenazas posibles y los mecanismos para contrarrestarlas.

Propiedades de comunicación segura:

- **Validez**: todo mensaje enviado será recibido.
- **Integridad**: todo mensaje recibido es correcto y respeta la secuencialidad.

Amenazas a procesos: acceso indebido, suplantación (spoofing), falsificación de servicios o peticiones.

Amenazas a canales: captura (eavesdropping), reenvío (replay attacks), eliminación o modificación de mensajes; ataques de denegación de servicio (DoS/DDoS).

Técnicas de seguridad:

- **Criptografía**: encriptación (privacidad), firmas digitales (autenticidad), autenticación (identidad), contratos digitales (legalidad).
- **Control de acceso**: a recursos y servicios (ACLs, RBAC).
- **Filtrado de red**: mecanismos de confianza por dominios, tunelización, VPN.
- **Canales seguros**: TLS/SSL para comunicación cifrada extremo a extremo.
- **Certificados digitales**: PKI (Public Key Infrastructure) para verificar identidades.

Principios de diseño seguro:

- **Mínimo privilegio**: cada componente tiene solo los permisos estrictamente necesarios.
- **Defensa en profundidad**: múltiples capas de seguridad; ninguna es el único punto de protección.
- **Fail-safe defaults**: ante duda, denegar acceso.
- **Separación de privilegios**: operaciones sensibles requieren múltiples autorizaciones independientes.